Master's Degree Project in Mathematics
(MAA044)

30 ECTS credits

Date:

June 4, 2022

Credit Scoring using Machine Learning Approaches

***Author*:**
Bornvalue Chitambira

***Supervisor*:**
Christopher Engström

DIVISION OF MATHEMATICS AND PHYSICS
MÄLARDALEN UNIVERSITY
SE-721 23, VÄSTERÅS, SWEDEN

MAA044 — Master's Degree Project in Mathematics

Date of presentation:
01 June 2022

Project name:
Credit Scoring using Machine Learning Approaches

Author:
Bornvalue Chitambira

Version:
June 4, 2022

Supervisor(s):
Christopher Engström

Examiner:
Daniel Andrén

Comprising:
30 ECTS credits

Abstract

This project will explore machine learning approaches that are used in credit scoring. In this study we consider consumer credit scoring instead of corporate credit scoring and our focus is on methods that are currently used in practice by banks such as logistic regression and decision trees and also compare their performance against machine learning approaches such as support vector machines (SVM), neural networks and random forests. In our models we address important issues such as dataset imbalance, model overfitting and calibration of model probabilities. The six machine learning methods we study are support vector machine, logistic regression, k-nearest neighbour, artificial neural networks, decision trees and random forests. We implement these models in python and analyse their performance on credit dataset with 30000 observations from Taiwan, extracted from the University of California Irvine (UCI) machine learning repository.

Keywords: Credit Scoring, Logistic Regression, Decision Trees, Artificial Neural Networks, Random forests, Support Vector Machine, k-nearest neighbour, cross validation, imbalanced dataset.

Acknowledgements

I would like to extend gratitude to my dear wife Neptune for the encouragement she has given me throughout my studies. My parents and siblings have been very supportive from the onset. A big thank you goes to Christopher Engström, my thesis supervisor, for unrelenting efforts he put in guiding me through my thesis writing process. Much appreciation is given to the Swedish Institute for granting me scholarship to study in Sweden. This rare opportunity afforded me access to remarkable and quality education at Mälardalen University, and meet with amazing colleagues during my study period. Above all, I give thanks to God for intellectual ability to carry out my studies. I could not have done any of this without strength from Him.

List of Tables

1.1 Example of Confusion matrix 10

2.1 Summary of Predictive Performance of six data mining techniques from Yeh and Lien [14] 14
2.2 Summary of Predictive Performance of three data mining techniques from Zhang et al [15] 15
2.3 Confusion Matrices of three data mining techniques from Zhang et al [15]. 15

4.1 Description of Dataset 35

5.1 Summary of Predictive Performance of six Credit Scoring Approaches (With Cross Validation) from our experiments 39
5.2 Confusion Matrices of six Credit Scoring Approaches (With Cross Validation) from our experiments. 39
5.3 Summary of Predictive Performance of six Credit Scoring Approaches (Without Cross Validation) from our experiments . . . 40
5.4 Brier Skills Score 42

List of Figures

1.1 Example of a Receiver Operating Characteristic Curve 10

3.1 Illustration of SVM in 2-dimensions 18
3.2 Multilayer Perceptron with one hidden layer 23
3.3 Transfer functions in Neural Networks 25
3.4 Entropy for 2-class problem 29
3.5 Random Forest for Classification 30

4.1 K-Fold Cross validation illustration 33

Contents

Acknowledgements **2**

List of Tables **3**

List of Figures **4**

1 Introduction **7**
1.1 Problem Statement 7
1.2 Background on Credit Scoring 7
1.3 Credit Scoring History and Developments 8
1.4 Notation 9
1.5 Definitions 10

2 Literature Review **12**

3 Machine Learning Methods **17**
3.1 Support Vector Machine 17
3.2 Logistic Regression 21
3.3 Artificial Neural Networks 23
3.3.1 Definition: The Perceptron 23
3.3.2 Multilayer Perceptron (MLP) 23
3.3.3 Issues to consider when training artificial neural networks 26
3.4 Classification Trees 28
3.5 Random Forests 30
3.6 K-nearest neighbours (kNN) 31

4 Machine Learning Models Implementation **32**
4.1 Modelling Considerations 32
4.1.1 Dataset Split 32
4.1.2 Stratified K-fold Cross Validation 32
4.1.3 Feature Transformation 34
4.1.4 Cost-sensitive classification 34
4.2 Dataset Description 34
4.3 Algorithms implementation 35

5 Results and Evaluation **37**
5.1 Results 38
5.2 Discussion of Results 40

6 Conclusions and Future Work **43**
6.1 Conclusions 43
6.2 Future Work 44
References 45
Appendix 47

Chapter 1

Introduction

1.1 Problem Statement

The advances in financial technology (fintech) has facilitated new ways of doing business in the financial services industry. Financial institutions involved in lending are now compelled to push their offerings closer to the market than before, be more transparent and non-discriminating in their credit underwriting and to reduce their loan application turnaround. This has resulted in the need for improvement in the methods that have been used to decide on whether to grant or not grant credit. Moreover with ongoing discussions by the European Banking Authority (EBA), the future of machine learning models is more prevalent. The question we still have to answer is that, of the models that have been used and the novel methods, which of them give the best results as far as discriminating between good and bad borrowers? Our thesis aims to study some of the widely used methods to analyse their use cases and shortcomings and then we implement the methods in python inorder to assess their performance.

1.2 Background on Credit Scoring

Credit scoring is a statistical method which is used to predict the probability that a loan applicant, existing borrower, or counterparty will default or become delinquent. It provides an estimate of the probability of default or delinquency, which is widely used for consumer lending, credit cards, and mortgage lending [8]. Credit scoring is used by lenders to decide whether to grant credit or not to borrowers who apply to them [2]. The goal of any credit scoring process is to be able to summarise all the borrower's available information into a score. If a borrower's score is found to be above a predetermined threshold credit is granted, otherwise it is denied. Credit scoring is applied both to new applicants in *application scoring* and to monitor existing borrowers so as to determine whether they are changes in their creditworthiness: *behavioural scoring*.

In both application scoring and behaviour scoring, it is important for lending providers to have a large sample of previous customers together with their application details and subsequent credit history [13]. It is from this large sample that the scoring techniques will establish connections between the characteristics of a consumer and how good or bad their credit history is. Each of the consumer's characteristics such as *income, number of years employed, age* will be given scores and the total of those scores constitute a scorecard which is used to determine whether the consumer will be a good borrower or not and hence assist the lender in making that decision to grant credit or not. It is also important to note that some of the methods to be studied in this thesis will not provide customer scores but will instead give a probability that the consumer will be delinquent.

1.3 Credit Scoring History and Developments

In [13], it is noted that in 1941, David Durand was the first to recognize that one could use the same techniques that Fisher used in 1936 when he sought to solve the problem of differentiating between groups of a population. Durand carried out a research project for the U.S. National Bureau of Economic Research though it was not for predictive purposes [13]. In the 1950s Bill Fair and Earl Isaac established a consultancy in San Francisco, which used statistically derived models in lending decisions. Credit scoring's usefulness became more apparent in the 1960s when credit cards were introduced. Following the success in credit cards, banks and other lending institutions in the 1980s started using scoring in other products. It is important also to note that the objective of credit scoring is to predict risk not to explain why some customers default. Like any other model, credit scoring should be founded on sound methodology and the data used in modelling should be empirically derived.

Statistical methods have been used since the 1950s and 1960s and are still popularly used today because they enable lenders to use concepts of sample estimators, confidence intervals and statistical inference, that is, hypothesis testing in credit scoring. This allows scorecard developers to evaluate discriminatory power of models and also to determine which borrower characteristics are more important in explaining borrower behaviour. Linear discriminant analysis was one of the earliest approaches to be used in credit scoring. Even though the scorecards it produced were very robust, the assumptions which were needed to ensure satisfactory discriminatory power were restrictive. According to [13], logistic regression has taken over from linear discriminant analysis and has been successful as the statistical method for credit scoring.

According to [8], the methods used for credit scoring have increased in sophistication in recent years. They have evolved from traditional statistical techniques to innovative methods such as artificial intelligence, including machine

learning algorithms such as random forests, gradient boosting, and deep neural networks. With the ongoing discussions in the banking industry, the future of machine learning (ML) models will soon be more prevalent [4]. According to a recent discussion paper [3] by the European Banking Authority, Machine Learning models applied in Basel II regulation's Internal Rating Based (IRB) approach can be beneficial in the following ways;

- Improving risk differentiation, both by improving the model discriminatory power and by providing useful tools for the identification of all relevant risk drivers or even relations among them. ML models might be used to optimise the portfolio segmentation, to build robust models across geographical and industry sectors/products and take data-driven decisions that balance data availability against the required model granularity. Moreover, ML models might help to confirm data features selected by expert judgement used for 'traditional' model development giving a data-driven perspective to the feature selection process

- Improving data collection and preparation processes including, for example, cleaning of input data or by providing a tool for data treatment and data quality checks. ML models might also be used for performing outlier detection and for error correction.

- Providing robust systems for validation and monitoring of the models. ML models might be used to generate model challengers or as a supporting analysis for alternative assumptions or approaches

- Performing stress testing, by assessing the effect of certain specific conditions on the total capital requirements for credit risk and by identifying adverse scenarios

1.4 Notation

1. A scalar is denoted in normal script $x \in \mathbb{R}$.

2. Vectors are represented in bold letters and are assumed to be column vectors as: $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$.

3. The row vectors are represented as transpose of the column vectors as: $\mathbf{x}^T = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}^T = \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix}$.

4. Matrices are represented by bold capital letters as: $\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n_h 1} & w_{n_h 2} & \dots & w_{n_h n} \end{bmatrix}$.

1.5 Definitions

Confusion Matrix is a table that is used to describe the performance of a classification model (or "classifier") on a set of test data for which the true values are known. Table 1.1 below shows an example of a Confusion matrix

		Predicted	
		Yes	No
Actual	Yes	True Positive	False Negative
	No	False Positive	True Negative

Table 1.1: Example of Confusion matrix

Receiver Operating Characteristic (ROC) Curve is a plot which illustrates the diagnostic ability of a binary classifier system as its discrimination threshold is varied. Figure 1.1 below shows an example of a ROC curve.

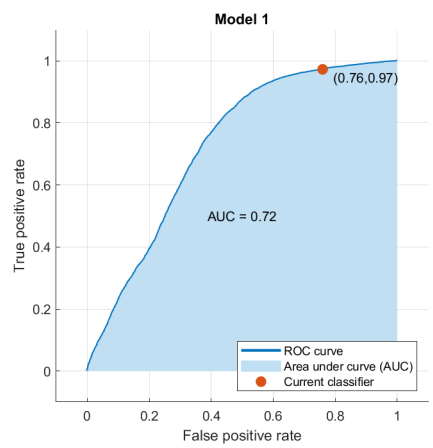

Figure 1.1: Example of a Receiver Operating Characteristic Curve

Accuracy is the ratio of number of correct predictions to the total number of input samples.

Precision refers to the number of true positives divided by the total number

of positive predictions.

Recall is a metric that quantifies the number of correct positive predictions made out of all positive predictions that could have been made.

F1-score is defined as the harmonic mean of precision and recall.

Chapter 2

Literature Review

In this section we explore the previous studies which have been conducted in credit scoring. We briefly look at some of the methods they investigated and the results they got. This will enable us to see the academic progression of the subject and also give context to our study and the comparison of our results thereof.

I-Cheng Yeh and Che hui Lien [14] carried out a study and compared six methods (data mining techniques) used in credit scoring: *K-nearest neighbors, Logistic regression, Discriminant Analysis, Naive Bayesian, Neural networks and Classification trees*, which are briefly introduced below.

K-nearest neighbours (KNN)

K-nearest neighbor (KNN) classifiers are based on learning by analogy whereby given an unknown sample, a KNN classifier carries out a search in the pattern space for the KNN that are closest to the unknown sample in terms of Euclidean distance:

$$d_{(i)} = ||x_{(i)} - x_0||.$$

The unknown sample is then assigned the most common class among its KNN [14]. For example, given a query point x_0, we find the k training points $x(i)$, $i = 1, ..., k$ closest in distance to x_0, and then classify using majority vote among the k neighbors. These classifiers are memory-based, and require no model to be fit [7]. The major advantage of this approach is that it is not required to establish predictive model before classification. The disadvantages are that KNN does not produce a simple classification probability formula and its predictive accuracy is highly affected by the measure of distance and the cardinality k of the neighborhood.

Logistic Regression

Logistic Regression is a special case of the linear regression. The model specifies that an appropriate function of the fitted probability of the event is a linear

function of the observed values of the available explanatory variables [14]. The major advantage of this approach is that it can produce a simple probabilistic formula of classification though it suffers weaknesses that it cannot properly solve problems non-linear problems and interactive effects of explanatory variables.

Discriminant Analysis (DA)

DA is an alternative to logistic regression and is based on the assumptions that, for each given class of response variable, the explanatory variables are distributed as a multivariate normal distribution with a common variance–covariance matrix. The objective of DA is to maximize the distance between different groups and to minimize the distance within each group [14].

Naive Bayes (NB)

The naive Bayesian classifier is based on Bayes theory and is premised around the class conditional independence assumption which asserts that the effect of an attribute value on a given class is independent of the values of the other attributes. Bayesian classifiers provide a theoretical justification for other classifiers that do not explicitly use Bayes theorem. The major weakness of NB is that the predictive accuracy is highly correlated with the assumption of class conditional independence. This assumption simplifies computation. In practice, however, dependences can exist between variables [14].

Artificial Neural Networks (ANN)

Artificial neural networks use non-linear mathematical equations to successively develop meaningful relationships between input and output variables through a learning process. Back propagation networks are applied to classify data. A back propagation neural network uses a feed-forward topology and supervised learning. The structure of back propagation networks is typically composed of an input layer, one or more hidden layers, and an output layer, each consisting of several neurons [14]. The advantage of ANNs is that they can easily handle the non-linear and interactive effects of explanatory variables. They however cannot result in a simple probabilistic formula of classification.

Classification Trees (CT)

Classification trees perform a classification of the observations on the basis of all explanatory variables and supervised by the presence of the response variable. The segmentation process is typically carried out using only one explanatory variable at a time. CTs are based on minimizing impurity, which refers to a measure of variability of the response values of the observations. CTs can result in simple classification rules and can handle the non-linear and interactive effects of explanatory variables [14].

In the study by I-Cheng Yeh and Che hui Lien [14], they took payment data with 25 000 observation from Taiwan cash and card issuer from October 2005, of which 22.12% of the cardholders had default payments. They used 23 variables as predictors. They used the "sorting smoothing method" to estimate the real probability of default and then applied linear regression $y = \alpha + \beta X$ to examine whether predicted probability of default represented real probability of default. In their method they concluded if the R^2 was close to 1, α was zero and the β was close to 1 then data mining method's predicted probability produced the real probability of default. It was observed that artificial neural networks produced the highest R^2 and the performance of the predictive power of all six methods are given in the following table 2.1 ;

Method	Regression Coefficient	Regression Intercept	Regression R^2
K-nearest neighbours	0.770	0.0522	0.876
Logistic regression	1.233	-0.0523	0.794
Discriminant Analysis	0.837	-0.1530	0.659
Naive Bayesian	0.502	0.0901	0.899
Neural Networks	0.998	0.0145	0.965
Classification trees	1.111	-0.0276	0.278

Table 2.1: Summary of Predictive Performance of six data mining techniques from Yeh and Lien [14]

The study found out that Artificial neural networks perform classification more accurately than the other five methods.

In 2018, Zhang et al [15] carried out a study on classification and prediction on Rural Property Mortgage data using three data mining methods, *Bayesian network (BN), Artificial Neural Network (ANN)* and *Logistic Regression (LR).* The objective of their study was to investigate the causal relationships among variables in rural property mortgage data and then build and evaluate the classification and prediction model for farmers loan demand under the three methods given above. The paper used data set with 4000 observations taken between July 2011 and July 2012 from Western China on rural farmers and 11 factors were used as predictor variables. These were categorised into 3 broad categories that is basic information, loan status and demand for property mortgage rights.

In their findings they noted that ANN recorded highest accuracy of 0.914 and is considered the best predictive method. Accuracy measures the performance of a model by dividing the number of classifications a model correctly predicts by the total number of predictions made. The results of their study are given in the Table 2.2;

	Accuracy	Sensitivity	Specificity
BN	0.880	0.750	0.934
ANN	0.914	0.769	0.973
LR	0.907	0.777	0.960

Table 2.2: Summary of Predictive Performance of three data mining techniques from Zhang et al [15]

	BN	
	Yes	No
Yes	437	146
No	94	1322

	ANN	
	Yes	No
Yes	448	135
No	410	1375

	LR	
	Yes	No
Yes	453	130
No	56	1360

Table 2.3: Confusion Matrices of three data mining techniques from Zhang et al [15].

Liu R, L [11] conducted research on credit card clients and compared *Support Vector Machine, k-Nearest Neighbors, Decision Tree* and *Random Forest,* with *Feedforward Neural Network* and *Long Short-Term Memory* in 2018. The study was carried out to refine earlier work which was done by Yeh [14]. Lui [11] proposed to add two important factors dropout and long short-term memory to neural networks inorder to find their effect on improving accuracy and also solving the problem of overfitting. They used the same dataset as in [14] and the 30,000 samples were randomly shuffled, after which the top 10,000 samples were chosen. The top 8500 samples were used as training set, with the other 1500 being used as testing set. The data was normalized to mean of 0 and variance of 1.

Of the traditional machine learning methods, that is *Support Vector Machine, k-Nearest Neighbors, Decision Tree* and *Random Forest*, the highest accuracy of 0.8040 was achieved by support vector machines. On the other hand the accuracy of neural networks was 0.8246 and in this research neural networks outperformed other methods. They also noted that for FeedForward Neural networks, using dropout can improve performance especially when number of neurons per layer is large. For Long Short-Term Memory it was noted that using dropout does not result in any significant difference in accuracy.

In [10] Stefan Lessmann et al looked at some of the improvements that could be introduced to credit scoring. Their study used 41 classification methods across

8 credit scoring data sets. Their key focus was on data, classification methods and indicators used to assess performance of the classification methods. They analysed popular credit scoring research in chronological order and found three important areas of improvement:

- Type of Credit data sets
  Most researches analysed were found to have used smaller number of data sets. They reasoned that using multiple data sets facilitates examining robustness of scorecards towards environmental conditions, hence their use of 8 data sets.

- Classification algorithms employed
  Second, they discovered that the number of classification methods used differed considerably per each study. They therefore proposed conducting a large-scale comparison of several established and novel classification algorithms including selective ensemble methods.

- Indicators used to assess performance
  Third, it was noted that most studies rely on a single performance measure or measures of the same type. They put performance measures into three categories: those that assess discriminatory ability of the scorecard (e.g., AUC), those that assess the accuracy of the scorecard's probability predictions (e.g., Brier Score) and those that assess the correctness of the scorecards' categorical predictions (e.g., classification error). Only few studies were found to mix evaluation measures from different categories. This misses an important aspect of scorecard performance because financial institutions require PD estimates that are not only accurate but also well calibrated. As a result they proposed the use of conceptually different performance criteria. They used six indicators to measure the predictive accuracy of a scorecard: the percentage correctly classified (PCC), the AUC, a partial Gini index (PG), the H-measure, the Brier Score (BS), and the Kolmogorov-Smirnov statistic (KS).

As part of their conclusion they selected and compared performance of four methods Logistic Regression, Artificial Neural Networks, Random Forests, and Hill-Climbing Bagged Ensemble Selection (HCES-Bag). Logistic Regression was selected for its popularity in credit scoring, and the other three for performing best in their category (best individual classifier, best homogeneous/heterogeneous ensemble). They also examined to what extent four selected classifiers were statistically different. They concluded that

1. Logistic Regression predicts significantly less accurately than any of the other classifiers,

2. HCES-Bag predicts significantly more accurately than any of the other classifiers,

3. and the empirical results did not provide sufficient evidence to conclude whether Random Forests and Artificial Neural Networks perform significantly different.

Chapter 3

Machine Learning Methods

3.1 Support Vector Machine

Support Vector Machine (SVM) are a generalization of a simple intuitive classifier called *maximum margin classifier* [7]. The maximum margin classifier requires the classes to be separable by a linear boundary and because of this limitation, the support vector classifier can be applied to broader range of cases. Support Vector Machine is a further extension of the support vector classifier, which is intended to address the problem of non-linear class boundaries. Support Vector Machine produces non-linear boundaries by constructing a linear boundary in a large transformed version of the feature space.

Support Vector Machine was first developed by Cortes and Vapnick (1995) for Binary Classification [5]. The concept attains the classification goal by finding an optimal separating hyperplane between the classes that maximizes the class margin. Figure 3.1 shows 2-dimensional separation case. The boundaries of the margin are represented by dotted lines and are labeled $\mathbf{w}^\top\mathbf{x} + b = 1$ and $\mathbf{w}^\top\mathbf{x} + b = -1$. The points lying on the boundaries of the margin are called *support vectors*. The middle margin represented by solid line and labelled $\mathbf{w}^\top\mathbf{x} + b = 0$ is the optimal separating hyperplane.

In credit scoring the decision to classify an applicant as "good" or "bad" borrower depends on the SVM score and this score is a linear combination of the consumer's predictive characteristics multiplied by some weights as in the following;

$$f(x) = w_1x_1 + w_2x_2 + ... + w_nx_n + b, \tag{3.1}$$

where n is the number of predictive characteristics, $w_i{}'s$ for $i = 1, ..., n$ and b are the learnt parameters and $x_i{}'s$ for $i = 1, ..., n$ are the consumer's predictive characteristics. Equation (3.1) can be represented neatly in form of vectors as:

$$f(x) = \mathbf{w}^\top\mathbf{x} + b. \tag{3.2}$$

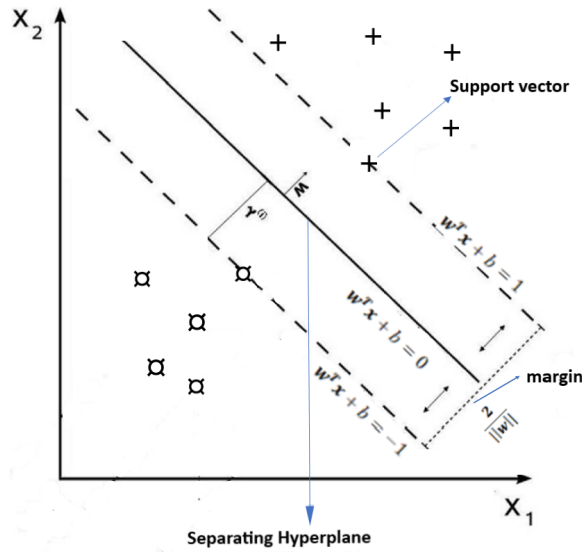

Figure 3.1: Illustration of SVM in 2-dimensions

The original training examples are $(\mathbf{x}_1, y_1), ..., (\mathbf{x}_n, y_n)$, where the $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{-1, 1\}$. These training examples are transformed to a high-dimensional space $\mathcal{H}$ using a mapping Φ to produce $(\Phi(\mathbf{x}_1), y_1), ..., (\Phi(\mathbf{x}_n), y_n)$, where $\Phi(\mathbf{x}_1) \in \mathcal{H}$ [9]. We then seek a hyperplane that separates the transformed training examples with maximal margin in the transformed space $\mathcal{H}$. To classify a new training example $\mathbf{x}$, we compute $\Phi(\mathbf{x})$ and classify $\mathbf{x}$ according to which side of the hyperplane $\Phi(\mathbf{x})$ lies. A hyperplane in this transformed space $\mathcal{H}$ can be represented in terms of a vector $\mathbf{w} \in \mathcal{H}$ and a scalar b. The equation of the hyperplane is given by;

$$\mathbf{w}^\top \mathbf{z} + b = 0,$$

where $\mathbf{z} \in \mathcal{H}$ and any point $\mathbf{z} \in \mathcal{H}$ that satisfies this equation lies on the hyperplane. Consequently all points that lie on one side of the hyperplane will satisfy $\mathbf{w}^\top \mathbf{z} + b > 0$ while all points lying on the other side of the hyperplane will satisfy $\mathbf{w}^\top \mathbf{z} + b < 0$ [9]. Inorder to find a separating hyperplane, we need to find $\mathbf{w}$ and b such that;

$$\mathbf{w}^\top \Phi(\mathbf{x}_i) + b > 0$$

if $y_i = +1$,

$$\mathbf{w}^\top \Phi(\mathbf{x}_i) + b < 0$$

if $y_i = 1$. In higher dimensions, the discrimination function becomes

$$f(x) = \mathbf{w}^\top \Phi(\mathbf{x}_i) + b. \tag{3.3}$$

To find the optimal hyperplane which achieves the maximum geometric margin,the following optimization problem is solved:

$$\begin{aligned} &\max_{\gamma,w,b} \gamma, \\ \text{subject to} \quad y_i(\mathbf{w}^\top \Phi(\mathbf{x}_i) + b) \geq \gamma, \quad & i = 1, .., n \\ ||\mathbf{w}|| = 1. \end{aligned} \tag{3.4}$$

Because constraint $||\mathbf{w}|| = 1$ is non-convex, the optimization problem (3.4) is transformed into (3.5). If $\hat{\gamma} = 1$, then $\hat{\gamma}/||\mathbf{w}|| = 1/||\mathbf{w}||$ and maximizing this is the same as minimizing $||\mathbf{w}||^2$. [5]

$$\begin{aligned} &\min_{\gamma,w,b} \frac{1}{2}||\mathbf{w}||^2, \\ \text{subject to} \quad y_i(\mathbf{w}^\top \Phi(\mathbf{x}_i) + b) \geq 1, \quad & i = 1, .., n. \end{aligned} \tag{3.5}$$

The problem formulated so far works for linearly separable classes. In real life credit datasets are not linearly separable so we would allow a classifier that misclassifies a few training observations so that it can do a better job in classifying the remaining observations [2]. A regularization term ξ and C penalty are therefore added to the optimization problem in (3.5) so that it would work for non-linearly separable problem

$$\begin{aligned} &\min_{w,b,\xi} \frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{n} \xi_i, \\ \text{subject to} \quad y_i(\mathbf{w}^\top \Phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad & i = 1, .., n \\ \xi_i \geq 0, \quad & i = 1, .., n. \end{aligned} \tag{3.6}$$

The positive constant C in (3.6) is a tuning parameter indicating the tradeoff between a small and large number of margin failures. A smaller C permits larger number of errors while a larger C permits only a few errors. The linearly separable case corresponds to $C = \infty$. The $\xi_1, ..., \xi_n$ are *slack variables* that allow individual observations to be on the wrong side of the hyperplane.

The solution to the optimization problem (3.6) is given by the saddle point of the Lagrangian function with constraints $\alpha_i's$ and $\mu_i's$. The problem can then be transformed into its dual form as;

$$\mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mu) = \frac{1}{2}||\mathbf{w}||^2 + C\sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \alpha_i[y_i(\mathbf{w}^\top \Phi(\mathbf{x}_i) + b) - 1 + \xi_i] - \sum_{i=1}^{n} \mu_i \xi_i. \tag{3.7}$$

Equation (3.7) is minimised with respect to w and b by taking partial derivatives and setting them equal to zero;

$$\frac{\partial}{\partial \mathbf{w}} \mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mu) = \mathbf{w} - \sum_{i=1}^{n} \alpha_i y_i \Phi(\mathbf{x}_i) = 0, \tag{3.8}$$

$$\frac{\partial}{\partial b}\mathcal{L}(\mathbf{w}, b, \xi, \alpha, \mu) = \sum_{i=1}^{n} \alpha_i y_i = 0. \tag{3.9}$$

Solving (3.8) for w produces

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \Phi(\mathbf{x}_i), \tag{3.10}$$

for some i, where $\alpha_i = 0$, the corresponding example $(\Phi(\mathbf{x}_i), yi)$ does not affect the maximum margin hyperplane. But for other i, where we have $\alpha_i > 0$, and the examples corresponding to these i's do affect the maximum margin hyperplane and these examples (corresponding to positive α_i) are the support vectors.

If we substitute the expression for **w** given in Equation (3.10) into the expression for the discriminant function in Equation (3.3), we get

$$\mathbf{w}^\top \Phi(\mathbf{x}_i) + b = \sum_{i=1}^{n} \alpha_i y_i (\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x})) + b. \tag{3.11}$$

Under certain conditions the dot product $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x})$ can be replaced with a kernel function $K(\mathbf{x}_i, \mathbf{x})$ that is easy to compute and the resulting classifier becomes a form of a general kernel classifier so that equation (3.11) becomes

$$\mathbf{w}^\top \Phi(\mathbf{x}_i) + b = \sum_{i=1}^{n} \alpha_i y_i (\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x})) + b = \sum_{i=1}^{n} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b. \tag{3.12}$$

A given feature vector x is classified as 1 if $\sum_{i=1}^{n} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b > 0$ and x is classified as -1 if $\sum_{i=1}^{n} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b < 0$ [9]. In practice, one generally directly chooses the kernel function K, while the mapping $\Phi()$ and the transformed space $\mathcal{H}$ are induced by the choice of K. This means that once we specify a kernel, the training and classification rule can be implemented directly without the need for even knowing what are the corresponding Φ and $\mathcal{H}$ [9]. The kernel function K is implemented so that instead of mapping two instances $\mathbf{x}_i$ and $\mathbf{x}$ to the z-space and doing a dot product there, we directly apply the kernel function in the original space [1].

3.2 Logistic Regression

In cases where a dependent variable falls into two categories, as in the case of customer default, logistic regression enables us to model the the probability that a response belongs to a particular category. It is a classification algorithm that works by trying to learn a function that approximates $P(Y|X)$ and it makes the central assumption that $P(Y|X)$ can be approximated as a sigmoid function applied to a linear combination of input features.

Given a training set of N data points $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with input data $\mathbf{x}_i \in \mathbb{R}^n$ and the associated binary class labels $y_i \in \{0, 1\}$, the logistic regression estimates the probability $P(y = 1|\mathbf{x})$ as:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + \exp\left[-(w_0 + \mathbf{w}^T\mathbf{x})\right]} \tag{3.13}$$

and

$$P(y = 0|\mathbf{x}) = \frac{\exp\left[-(w_0 + \mathbf{w}^T\mathbf{x})\right]}{1 + \exp\left[-(w_0 + \mathbf{w}^T\mathbf{x})\right]}, \tag{3.14}$$

where $\mathbf{x}_i \in \mathbb{R}^n$ is n-dimensional input vector, $\mathbf{w}$ is the parameter vector and the scalar w_0 is the intercept [2].

In order to chose values for the parameters $\mathbf{w}$ and w_0 of logistic regression, we use maximum likelihood estimation. The labels that we are predicting are binary, and the output of our logistic regression function is supposed to be the probability that the label is one. This means that we can (and should) interpret each label as a Bernoulli random variable: $Y \sim \mathrm{Ber}(p)$ where $p = \sigma(\mathbf{w}^T\mathbf{x})$.

By the probability mass function of Bernoulli, the probability of observing either class is

$$p(y|x) = P(y = 1|\mathbf{x})^y (1 - P(y = 1|\mathbf{x}))^{1-y}. \tag{3.15}$$

Given that the sample is independently drawn, from (3.15) we have the following likelihood function

$$\prod_{i=1}^{N} P(y_i = 1|\mathbf{x}_i)^{y_i} (1 - P(y_i = 1|\mathbf{x}_i))^{1-y_i}, \tag{3.16}$$

and the log-likelihood function becomes

$$LL = \sum_{i=1}^{N} y_i \log(P(y_i = 1|\mathbf{x}_i)) + (1 - y_i) \log(1 - P(y_i = 1|\mathbf{x}_i)). \tag{3.17}$$

The log-likelihood function can be maximised using Newton-Rhapson method [6]. Maximizing the log-likelihood is the same as minimizing the negative log-likelihood, which is also referred to as cross-entropy error function.

Since

$$\log\left(\frac{P(y=1|\mathbf{x})}{(1-P(y=1|\mathbf{x})}\right) = w_0 + \mathbf{w}^T\mathbf{x}, \tag{3.18}$$

the logistic regression model assumes a linear decision boundary modelled by the hyperplane $\{\mathbf{x}|w_0 + \mathbf{w}^T\mathbf{x} = 0\}$ [2].

Logistic regression suffers the problem of complete separation. If there is a feature that can completely separate the classes, logistic regression cannot be successfully trained. The weight of that feature will not converge and the optimal weight will be infinite.

3.3 Artificial Neural Networks

3.3.1 Definition: The Perceptron

An artificial neural network (ANN) model involves constructing mathematical representations that mimic certain processing capabilities of the human brain. The *perceptron* is the basic processing element which has its inputs coming from the environment or from the output of other perceptrons [1]. We define an input variable as $x_j \in \mathbb{R}$ for $j = 1, ..., d$. In the context of credit scoring. The input variables can be be attributes such as the applicant's income, debt, and the monthly payments out of the applicant's income. Associated with each input defined above is a *synaptic weight*, $w_j \in \mathbb{R}$ and the output z. The output of the network is the solution to a problem. For example, in this study, the output can be classification of "good" or a "bad" borrower.

3.3.2 Multilayer Perceptron (MLP)

A perceptron that has a single layer of weights suffers limitations in that it can only approximate linear functions of the input and cannot solve problems where the discriminant to be estimated is nonlinear. This limitation does not apply hidden layers to feedforward networks with intermediate or hidden layers between the multilayer input and the output layers. Therefore if multilayer perceptrons are used for classification, they can implement nonlinear discriminants.

A multilayer perceptron is composed of an input layer and one or more hidden layers, each with several neurons. Each of the neurons processes its inputs and produces one output value which it passes to the subsequent layer. Figure 3.2 shows the architecture of a multilayer perceptron with one hidden layer and one output neuron.

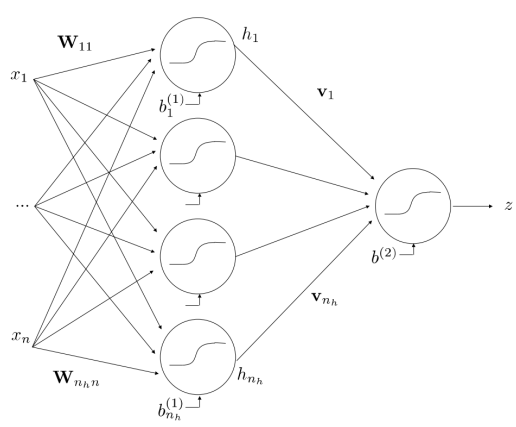

Figure 3.2: Multilayer Perceptron with one hidden layer

In Figure 3.2, x_j for $j = 1, .., n$ are the inputs, h_i is the output of the hidden layer and $\mathbf{W}$ is the weight matrix where $\mathbf{W}_{ij}$ denotes the weight of input j to the hidden unit i.

The output of h_i, the hidden layer is computed by processing the weighted inputs and the bias b_i as

$$h_i = f^{(1)}(b_i^{(1)} + \sum_{j=1}^{n} \mathbf{W}_{ij} x_j). \tag{3.19}$$

In the same way we process the output of z, the output layer as follows:

$$z = f^{(2)}(b_i^{(2)} + \sum_{j=1}^{n_h} \mathbf{v}_j h_j). \tag{3.20}$$

In equation (3.20), n_h is the number of hidden neurons and $\mathbf{v}$ is the weight vector where $\mathbf{v}_j$ is the weight connecting the hidden unit j to the output neuron. The bias units are the intercepts which make the model more general. By applying a threshold function, we can then map the neural network output into a classification problem.

The transfer functions also called *activation* functions, $f^{(1)}$ and $f^{(2)}$ are responsible for transforming the summed weighted input from the nodes into the activation of the nodes or output for those inputs. In general the simplest activation function is referred to as the linear activation, where no transform is applied at all. A network comprised of only linear activation functions is very easy to train, but cannot learn complex mapping functions. The transfer functions $f^{(1)}$ and $f^{(2)}$ in (3.19) and (3.20) therefore allow the neural network to model non-linear relationships in the data [2]. Figure 3.3 shows some different activation functions that can be used in neural networks.

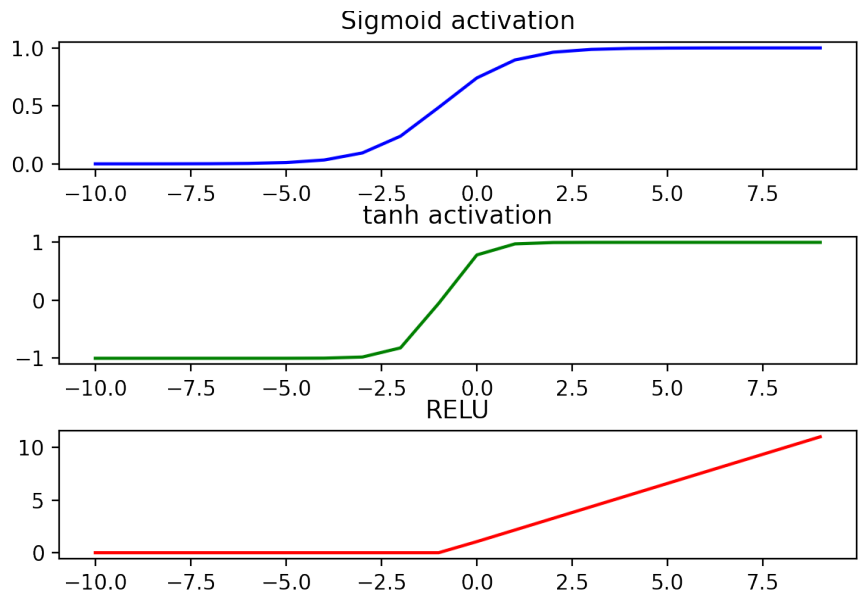

Figure 3.3: Transfer functions in Neural Networks

For binary classification problem in credit scoring, the most convenient activation function for the output layer, $f^{(2)}$ is the sigmoid function $\left(\frac{1}{1+e^{-x}}\right)$ since its output is bounded between $[0, 1]$. The output of z therefore becomes conditional probability $P(y = 1|\mathbf{v})$. That way the neural network produces a probability between 0 and 1 of a customer defaulting on their credit. For other applications it is worth noting that both hyperbolic tangent ($\tanh x$) and sigmoid functions cannot be used in networks with many layers due to the vanishing gradient problem. In such cases the rectified linear (ReLU) activation $\max(0, x)$ is the default activation since its performs faster and better.

The process of learning in a neural network involves computing output, and adjusting the weights in proportion to the difference between the outputs and the desired targets, and repeating the process. By repeating the learning process a number of times, the network identifies the correct values for the weights. The difference between the actual and desired output for a given set of inputs is an error called delta. The objective is to minimize the delta [12]. This algorithm is known as *backpropagation*. After each foward pass through the network, backpropagation performs a backward pass while adjusting the model's parameters.The algorithm repeatedly adjusts the weights and biases inorder to minimize a measure of the difference between the actual output vector and the target (desired) vector.

The level of adjustment of parameters in backpropagation depends on the gra-

dients of the cost function with respect to those parameters.

The parameters of the neural network $\mathbf{W}$ and $\mathbf{v}$ are estimated during a learning process. Given a training set of N data points $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, with input data $\mathbf{x}_i \in \mathbb{R}^n$ and corresponding binary class label $y_i \in \{0, 1\}$, the weights are first randomly initialized and then iteratively adjusted so as to minimize the objective function, which is the cross-entropy function as follows;

$$G = -\sum_{i=1}^{N} \{y_i \log(z_i) + (1 - y_i) \log(1 - z_i)\}. \tag{3.21}$$

We note that the above error function reaches its minimum when $z_i = y_i$ for all $i = 1, .., N$. It is also worth noting that logistic regression is a simplified neural network with one neuron and uses sigmoid function as its activation function and trained to minimize cross-entropy error. Stochastic Gradient descent derivation for backpropagation is detailed in [1]. The procedure uses Chain Rule for differentiation to calculate weight updates on the equation

$$\frac{\partial G}{\partial w_{ij}} = \frac{\partial G}{\partial z_i} \frac{\partial z_i}{\partial h_j} \frac{\partial h_j}{\partial w_{ij}}.$$

The update equations are as follows

$$\Delta v_j = \eta \sum_i (y_i - z_i) h_j \tag{3.22}$$

and

$$\Delta w_{ij} = \eta \sum_i (y_i - z_i) v_i h_i (1 - h_i) x_j, \tag{3.23}$$

where η is a constant.

3.3.3 Issues to consider when training artificial neural networks

- Overfitting (regularization and early stop)

  Often Neural networks with too many weights will overfit the data at the global minimum of the error and generalization will not be good. To curb this problem we train the model only for a while, and stop well before we approach the global minimum and then use a validation dataset to determine when to stop [6]. Methods like weight decay and weight elimination are employed.

- Initial values

  When randomly choosing values for weights, proper care has to be taken because choosing zero values will result in the algorithm not moving at all and hence achievement of perfect symmetry and zero derivatives. At the

same time choosing relatively larger values for weights will lead to poor results. The best choice is to begin with small values that are near zero so that the algorithm starts off linearly and then becoming nonlinear as the weights increase.

- Scaling of inputs

  Given the use of small weights in the model and the use of error between predictions and expected values, the scale of inputs and outputs used to train the model are an important factor. Unscaled input variables can result in a slow or unstable learning process. A good rule of thumb is that input variables should be small values, probably in the range of 0-1 or standardized with a zero mean and a standard deviation of one.

- Number of hidden layers and complexity

  Generally speaking it is better to have too many hidden units than too few. With too few hidden units, the model might not have enough flexibility to capture the non-linearities in the data; with too many hidden units, the extra weights can be shrunk toward zero if appropriate regularization is used [6]. Choice of the number of hidden layers is guided by background knowledge and experimentation.

3.4 Classification Trees

A decision tree is a hierarchical data structure implementing the divide-and-conquer strategy. It is an efficient nonparametric method, which can be used for both classification and regression [1]. For modeling problems where the target variable is classification outcome, a decision tree that is employed is known as the classification tree. When predicting an instance, we assign an observation in a given region to the most commonly occurring class of training observations in that region. That assignment may result in error of misclassification and the classification error rate is therefore the fraction of the training observations in that region that do not belong to the most common class [7]. The misclassifications can be quantified by the *node impurity measure*- $Q_m(T)$, which measures the homogeneity of the classes at a node. $Q_m(T)$ quantifies the goodness of a split. A split is pure if after the split, for all branches, all the instances choosing a branch belong to the same class [1].

In a node m, representing a region R_m with N_m observations,

let

$$\hat{p}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i = k),$$

be the proportion of class k observations in node m. According to the above, we classify the observations in node m to class $k(m) = \text{argmax}_k \hat{p}_{mk}$, the majority class in node m. Different measures of impurity are used as follows;

1. Misclassification error:

$$E = 1 - \hat{p}_{mk} = \frac{1}{N_m} \sum_{x_i \in R_m} I(y_i \neq k(m)). \tag{3.24}$$

   However, it turns out that misclassification error is not sufficiently sensitive for tree-growing, and in practice two other measures presented below are preferable [7]. If the prediction accuracy of a final pruned tree is the goal, then misclassification error can be preferred.

   In binary classification, for example in credit scoring where there are two classes (default or no default), the misclassification error becomes

$$E = 1 - \max(p, 1 - p).$$

2. Gini Index:

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}). \tag{3.25}$$

   Gini is a measure of total variance across the K classes. For that reason the Gini index takes on a small value if all of the $\hat{p}_{mk}$'s are close to zero

or one. A small value for Gini Index indicates that a node contains predominantly observations from a single class.

In binary classification, for example in credit scoring where there are two classes (default or no default), the gini index becomes

$$G = 2p(1-p).$$

3. Cross-entropy or deviance:

$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log(\hat{p}_{mk}). \tag{3.26}$$

Entropy measures how informative an attribute is in splitting data. Either cross-entropy or gini index are used in pruning a decision tree.

In binary classification, for example in credit scoring where there are two classes (default or no default), cross-entropy becomes

$$D = -p \log p - (1-p) \log(1-p).$$

In classification trees binary splits are preferred to multi-way splits because the latter fragments data too quickly, leaving insufficient data for the next level downwards [6].

For 2-class problem, entropy $Q(p, 1-p)$, where $p_1 = p$ and $p_2 = 1-p$, is a non-negative function satisfying the following properties [1] ;

- $Q(\frac{1}{2}, \frac{1}{2}) \geq Q(p, 1-p)$ for any $p \in [0, 1]$,
- $Q(0, 1) = Q(1, 0) = 0$, and
- $Q(p, 1-p)$ is increasing in p on $[0, \frac{1}{2}]$ and decreasing in p on $[\frac{1}{2}, 1]$.

Figure 3.4 shows the properties itemized above for a 2-class problem.

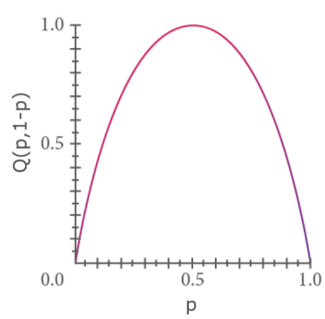

Figure 3.4: Entropy for 2-class problem

3.5 Random Forests

Random forests are substantial modification of bagging (bootstrap aggregating) that builds a large collection of de-correlated trees, and then averages them. Random forests are simpler to train and tune. As a consequence, random forests are popular, and are implemented in a variety of packages.

When building a random forest a number of decision trees are built on bootstrapped training samples. Bootstrapping is a resampling method that uses random sampling with replacement. Each time a split in a tree is considered, a random sample of m predictors is chosen as split candidates from the full set of p predictors [7]. In other words the algorithm is not allowed to consider all predictors, not even the majority when splitting. A fresh sample of m predictors is taken at each split and typically $m \approx \sqrt{p}$ is used. The reason for this is to avoid splits for most trees starting at the strongest predictor, leading to correlated trees. For bagging to result in reduction in variance, there is need for random forests to only consider a subset of predictors.

Figure 3.5 shows the algorithm of random forest for classification.

Algorithm *Random Forest for Classification.*

1. For $b = 1$ to B:
   (a) Draw a bootstrap sample $\mathbf{Z}^*$ of size N from the training data.
   (b) Grow a random-forest tree T_b to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size n_{min} is reached.
      i. Select m variables at random from the p variables.
      ii. Pick the best variable/split-point among the m.
      iii. Split the node into two daughter nodes.
2. Output the ensemble of trees $\{T_b\}_1^B$.

To make a prediction at a new point x:

Classification: Let $\hat{C}_b(x)$ be the class prediction of the bth random-forest tree. Then $\hat{C}_{\text{rf}}^B(x) = \textit{majority vote } \{\hat{C}_b(x)\}_1^B$.

Figure 3.5: Random Forest for Classification

3.6 K-nearest neighbours (kNN)

The nearest-neighbor method is a standard non-parametric approach to the classification problem first suggested by Fix and Hodges (1952). It was applied in the credit-scoring context first by Chatterjee and Barcun (1970) and later by Henley and Hand (1996) [13]. The nearest neighbours approach chooses a metric on the space of application data to measure how far apart any two applicants are. Using a sample of past applicants as a representative standard, a new applicant is then classified as good or bad depending on the proportions of "goods" and "bads" among the k-nearest applicants from the representative sample—the new applicant's nearest neighbors.

The three important parameters here are the metric d, how many applicants k constitute the set of nearest neighbors, and what proportion of these should be good for the applicant to be classified as good, that is the majority. If the majority of the neighbours are bad then then applicant is classified as bad. The choice of k can be made through cross-validation. It is also important to normalize inputs when using the kNN algorithm as items may have been measured in different units.

The choice for the metric d is usually the Euclidean distance;

$$d(\mathbf{x}_i, \mathbf{x}_j) = ||\mathbf{x}_i - \mathbf{x}_j|| = [(\mathbf{x}_i - \mathbf{x}_j)^\top (\mathbf{x}_i - \mathbf{x}_j)]^{1/2}, \tag{3.27}$$

where $\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^n$ are input vectors of data instance i and j respectively.

Chapter 4

Machine Learning Models Implementation

4.1 Modelling Considerations

All the models except ANN were trained and evaluated in *Scikit-Learn* which is a free software machine learning library for the Python programming language. We selected this platform not only because of it being open-source, but also for its generality of application to different types of algorithms and this makes it useful for comparing entirely different types of machine learning models against each other. We decided to train ANN in *Keras* because the library allowed us to treat imbalance in the distribution of the labels. The following considerations were addressed in the models we trained.

4.1.1 Dataset Split

Not all the data was used for training the models. The data was split into two parts and the part which was used for training is referred to as the training set. The other part was set aside for evaluation of the models is referred to as the test set. The Training and Test split choice takes into consideration of the amount of data we had. The aim of the split was to considerably balance between parameter estimates variance and performance statistic variance. Instead of the *train_test_split* module we used the *StratifiedKFold* module described below to perform dataset split.

4.1.2 Stratified K-fold Cross Validation

Here we split data into Training and Validation set, after setting aside a part of it as Test Set. In this method the data is divided randomly into k subsets of equal sizes. The holdout method is repeated k times, such that each time, one of the k subsets is used as the test set/validation set and the other $k-1$ subsets

are put together to form a training set. The error of estimation is averaged over all k trials to get total effectiveness of our model. By so doing every data point will be in a validation set exactly once, and gets to be in a training set $k-1$ times. This significantly reduces bias as we are using most of the data for fitting, and also significantly reduces variance as most of the data is also being used in validation set.

We used 10 Fold cross validation in our experiments, with details as follows;

Letting $\kappa : \{1, 2, ..., N\} \longmapsto \{1, 2, ..., K\}$ to be an indexing function which indicates the partition to which observation i is allocated by the randomization, we denote the fitted function by $\hat{f}^k(x)$, computed with the kth part of the data removed. The cross-validation estimate of prediction error is therefore

$$CV(\hat{f}) = \frac{1}{N} \sum_{i=1}^{N} L(y_i, \hat{f}^{-k(i)}(x_i)) \quad .$$

Figure 4.1 illustrates K-Fold cross-validation.

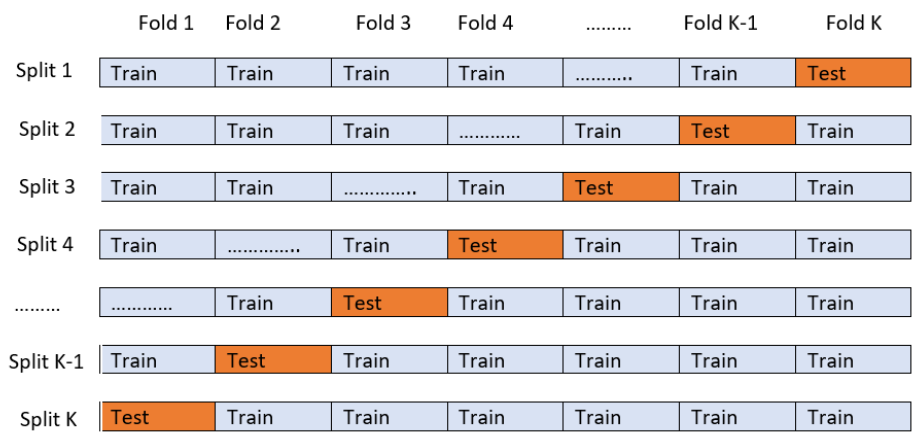

Figure 4.1: K-Fold Cross validation illustration

We employed Scikit-Learn's *StratifiedKFold* as a variation for our K-fold cross-validation. Stratified K-Fold ensures maintenance of proportions of labels in each of the folds and this is especially important given that we have an imbalanced dataset.

4.1.3 Feature Transformation

Algorithms that exploit distances and similarities between data samples such as *kNN*, *SVM* are sensitive to feature transformations and as such they might behave badly if the features do not look normally distributed. Other algorithms such as the graphical based classifiers are invariant to feature scaling, but it is good to standardise the data.

We used Scikit-Learn's *Standard Scaler*, which carries out normalization, that is, making the values in that data have $\mu = 0$ and $\sigma = 1$ in the following way;

$$x_i^{norm} = \frac{x_i - \bar{x_i}}{\sigma_i} \quad ,$$

where x_i is the original feature, $\bar{x_i}$ is the mean of variable x_i and σ_i, its standard deviation.

4.1.4 Cost-sensitive classification

Cost-sensitive classification methods have been developed to take into account the unequal misclassification costs involved in false positive and false negative errors by classification models, which are inherent to a variety of classification tasks. In credit scoring, incorrectly classified good and bad loan applicants incur different costs. A misclassified good loan applicant will be rejected and hence involves an opportunity cost, i.e. missed revenues, whereas a misclassified bad loan applicant will be accepted, potentially yielding a significant financial loss.

Scikit-Learn has the *class_weight* hyper-parameter, which penalizes mistakes in samples of class[i] with class_weight[i]. Higher class-weight means putting more emphasis on a class. We used the 'balanced' method which is a heuristic method that estimates class weights.

4.2 Dataset Description

The dataset used in this study was obtained from the UCI Machine Learning repository: `https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients`. The data was initially collected for research aimed at the case of customer default payments in Taiwan in October 2005. Among the total 30,000 observations, 6636 observations (22.12%) were the cardholders with default payment. This research employed a binary variable – default payment (Yes = 1, No = 0), as the response variable and used 23 variables as explanatory variables. Table 4.1 below gives the summary of the attributes, variable type and the corresponding categories for categorical variables.

The features are discriminated into numerical and categorical variables are *label encoded*, whereby a category is replaced by a numerical value. The dataset contains no missing values.

Attribute	Attribute ID	Variable Type	Categories
Amount of Given Credit	X1	Numerical	
Gender	X2	Categorical	1- Male 2-Female
Education	X3	Categorical	1-Graduate School 2-University 3-High School 4-Others
Marital Status	X4	Categorical	1-Married 2-Single 3-Others
Age	X5	Numerical	
Repayment status-Sep 2005	X6	Categorical	-1-Pay Duly 1-Payment Delay of 1 month ⋮ 9-Payment Delay of 9 months
Repayment status in August 2005	X7	Categorical	(Same as above)
Repayment status in July 2005	X8	Categorical	(Same as above)
Repayment status in June 2005	X9	Categorical	(Same as above)
Repayment status in May 2005	X10	Categorical	(Same as above)
Repayment status in April 2005	X11	Categorical	(Same as above)
Bill Statement-Sep 2005	X12	Numerical	
Bill Statement-Aug 2005	X13	Numerical	
Bill Statement-Jul 2005	X14	Numerical	
Bill Statement-Jun 2005	X15	Numerical	
Bill Statement-May2005	X16	Numerical	
Bill Statement-Apr 2005	X17	Numerical	
Amount paid in September 2005	X18	Numerical	
Amount paid in August 2005	X19	Numerical	
Amount paid in July 2005	X20	Numerical	
Amount paid in June 2005	X21	Numerical	
Amount paid in May 2005	X22	Numerical	
Amount paid in April 2005	X23	Numerical	

Table 4.1: Description of Dataset

4.3 Algorithms implementation

We implemented *Cost-sensitive Logistic Regression* in Scikit-Learn by applying the class-weight argument in the Logistic Regression method. The weighting penalized the model less for errors made on examples from the majority class and penalized the model more for errors made on examples from the minority class. The effect of the weights was to modify the cross entropy error function so that the optimization algorithm that minimizes the negative log likelihood

became

$$\min \sum_{i=1}^{n} -(w_0 \log(\hat{y_i}) y_i + w_1 \log(1 - \hat{y_i})(1 - y_i)) \ . \tag{4.1}$$

We trained *Cost-sensitive Classification Tree* using Sklearn's *DecisionTreeClassifier* module. We performed Grid Search to find the optimal parameters which would not lead the tree to overfit. We evaluated the tree classifiers using Area Under the Receiver Operating Characteristic Curve (AUROC) and the we retained the one which produced best parameters. This is referred to as pre-pruning. We found that the function for quality split was entropy with a maximum depth of 6. We then fitted the classification tree to the data based on the optimal hyper-parameters obtained.

For *Cost-sensitive Random Forest*, we carried out Grid Search on Scikit's *RandomForestClassifier* to find optimal hyper-parameters that give the best AUROC score. The random forest which produced the best parameters was trained with 250 trees, with maximum features of 3 and used bootstrap sampling. We then trained the final RF model based on the tuned hyper-parameters.

The *Cost-sensitive ANN* algorithm was implemented using Keras module in Python which provides support for class weighting. The fit() function that is used to train Keras neural network models takes an argument called class weight. This argument allows you to define a dictionary that maps class integer values to the importance to apply to each class.

Cost-sensitive Support Vector Machine was trained in Scikit-Learn's SVM.SVC module. As with the other classifiers, we performed Grid Search on the Support Vector Machine classifiers and retained the one with the best parameters. The method retained a set of parameters with the radial basis kernel function (rbf)

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma ||\mathbf{x} - \mathbf{x}'||^2\right),$$

where γ is specified by the parameter gamma in the listing 4.1. The regularization parameter C in equation (3.7) was chosen to be 3.0.

Listing 4.1: SVM Tuned Parameters

```
from sklearn.svm import SVC
svm=SVC(C=3.0,gamma='scale',kernel='rbf',random_state=0,class_weight={0:
    0.2212, 1: 0.7788})
```

For *k-Nearest Neighbour* however we did not perform cost sensitive learning, we used sklearn's *imblearn* module and we performed Synthetic Minority Oversampling Technique (SMOTE) to address the data imbalance. Consequently the algorithm produced fewer misclassifications of the minority class.

Chapter 5

Results and Evaluation

In this chapter we compare the different methods discussed in this report. We use widely used machine learning models performance metrics to evaluate the performance of the models. We report the following metrics;

- Accuracy

  Accuracy is a statistical measure of how well a binary classification test correctly identifies or excludes a condition, in our case default. It is the proportion of correct predictions (both true positives (non default) and true negatives(default)) among the total number of cases examined. The formula for accuracy is

  $$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} ,$$

  where TP-True Positives (True non-defaults), FP-False Positives (False non-defaults), TN-True Negatives(True default) and FN-False Negative(False defaults).

- Precision

  Precision is intuitively the ability of a classifier not to label as positive a sample that is negative. In other words precision aims to minimize false positives. Precision is given by the following formula,

  $$Precision = \frac{TP}{TP + FP} .$$

- Recall

  Recall measure answers the question, Among those instances that were classified as positive, what is the proportion that was missed? Recall

therefore seeks to minimize false negatives (false non-defaults). Recall is given by the following formula,

$$Recall = \frac{TP}{TP + FN} \ .$$

- F1 score

  F1 score is the harmonic mean of the precision and recall. It is given by the following formula,

$$F1 = 2 \ \cdot \frac{Recall \cdot Precision}{Recall + Precision} \ .$$

- AUROC

  An ROC curve (or receiver operating characteristic curve) is a plot that summarizes the performance of a binary classification model on the positive class. The x-axis indicates the False Positive Rate and the y-axis indicates the True Positive Rate.The AUROC depicts the skill of a classifier, that is its ability to discriminate between defaulters and non-defaulters. A no-skill classifier has AUROC = 0.5 while a perfect skill classifier has AUROC = 1.

- Confusion Matrix

  A confusion matrix provides visualization of performance of a classifier algorithm. More importantly it displays label noise, that is, examples from one class that are labelled as belonging to another class. This matrix provides a quick diagnostic of the performance of a classifier.

5.1 Results

In table 5.1 below, we present the results obtained from our implementation of the six credit scoring approaches using cross validation. Table 5.3 shows credit scoring techniques without cross-validation.

	Accuracy	Precision	Recall	F1	AUROC
Decision Tree (DT)	0.798	0.802	0.983	0.883	0.561
Artificial Neural Network (ANN)	0.765	0.883	0.794	0.835	0.713
Logistic Regression (LR)	0.815	0.839	0.943	0.888	0.654
Support Vector Machine (SVM)	0.808	0.846	0.920	0.881	0.665
Random Forest (RF)	0.813	0.845	0.931	0.886	0.664
K-nearest Neighbour (KNN)	0.754	0.851	0.831	0.840	0.658

Table 5.1: Summary of Predictive Performance of six Credit Scoring Approaches (With Cross Validation) from our experiments

Decision Tree

	Yes	No
Yes	2239	40
No	570	93

Artificial Neural Network

	Yes	No
Yes	1855	481
No	246	418

Logistic Regression

	Yes	No
Yes	2204	133
No	422	242

Support Vector Machine

	Yes	No
Yes	2150	186
No	391	272

Random Forest

	Yes	No
Yes	2173	164
No	398	266

K-nearest Neighbour

	Yes	No
Yes	1942	395
No	342	322

Table 5.2: Confusion Matrices of six Credit Scoring Approaches (With Cross Validation) from our experiments.

	Accuracy	Precision	Recall	F1	AUROC
DT	0.72	0.88	0.74	0.81	0.694
ANN	0.77	0.88	0.81	0.84	0.70
LR	0.68	0.87	0.69	0.77	0.672
SVM	0.78	0.88	0.84	0.86	0.71
RF	0.82	0.84	0.95	0.89	0.654
KNN	0.70	0.87	0.72	0.79	0.669

Table 5.3: Summary of Predictive Performance of six Credit Scoring Approaches (Without Cross Validation) from our experiments

5.2 Discussion of Results

In our experiments we applied 10-Fold Cross-validation. The Area under the ROC curve for most of the models without cross-validation show higher performance, but because we aimed to produce models that are fit for use-thereby having minimum model risk, we focused on the results from cross-validated models. We do not naively select models based on higher performance metrics, but we also need to consider other issues such as the ability of models to generalize. In credit scoring it is important to make credit decisions on models that do not overfit, and this is the reason why cross-validated models are preferred.

Based on Accuracy, *random forest* performed best followed by *logistic regression*, *support vector machine*, *decision tree*, *artificial neural network* and the lowest performance was reported by *kNN*. This result is consistent with other results obtained in previous research, for example in [14]. For our context however, we need other metrics to evaluate the models given the fact that our data is imbalanced.

Since our objective is to have a model that minimizes misclassifications of bad borrowers as good, Precision gives a better indicator of model performance in this regard. *Artificial neural network* has the highest precision of 0.884, followed by *support vector machine*. The least perfoming classifiers based on precision are *random forest* and *kNN*.

The Area Under the ROC curve shows that *artificial neural network* performs best compared to the other models in separating the good applicants from the bad applicants. Since our dataset is imbalanced the AUROC can be viewed as an objective performance measure. Even though we use it as a diagnostic tool, its use is limited when comparing different classifiers. *Decision tree* recorded the lowest AUROC of 0.561

We also used Brier Skill Score, which is one of the performance metrics proposed in a study by [10]. Brier skill score is a useful metric in comparing different classifiers against a benchmark. We used a no-skill classifier as the benchmark in our experiment. Brier Skill Score is derived as follows;

$$BrierScore(BS) = \frac{1}{N}\sum_{i=1}^{N}(f_t - O_t)^2,$$

where f_t is the probability that was forecast, O_t the actual outcome of the event at instance t and N is the number of forecasting instances. In effect, it is the mean squared error of the forecast. Brier Score is an appropriate metric for measuring accuracy of probabilistic predictions but it is not suitable for comparing models. To compare our models we used Brier Skill Score which is computed as

$$BrierSkillScore = 1 - \frac{BS_{model}}{BS_{benchmark}}.$$

The brier skill score can be more interpretable than the brier score because the BSS is simply the percentage improvement in the BS compared to the reference model, and a negative BSS means you are doing even worse than the reference model, which may not be obvious from looking at the brier score itself.

The following table 5.4 shows the Brier Skills Score

Model	Brier Skill Score
DT	0.855
ANN	0.565
LR	0.858
SVM	0.860
RF	0.859
KNN	0.818

Table 5.4: Brier Skills Score

Support Vector Machine gives the highest score, followed by *random forest*. *ANN* gives the least performance.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

In this thesis we examined machine learning techniques used for classification of credit applicants. We studied the mathematical foundations underpinning those methods. The six methods covered are *Logistic Regression*, *Decision Tree*, *Support Vector Machine*, *Artificial Neural Network*, *Random Forest* and *k-nearest neigbour*. Since dataset used for the problem was imbalanced, we applied cost sensitive learning to the algorithms for them to penalize more the misclassifications of the minority class. Useful techniques such as cross validation were also considered in our work to address model overfitting.

In our work, we tuned model hyperparameters inorder to address the issues explained above. For example we implemented grid search to pre-prune random forest and decision tree to avoid overfitting. For support vector machine we chose a moderate value for the penalty parameter C to allow for moderate errors in our algorithm. As a result of these adjustments, our results may appear biased towards our goal. There is possibility therefore for one to obtain different results if different hyperparameters are considered.

We also calibrated the models since uncalibrated probabilities usually are either over-confident or under-confident in their scoring. The calibrated probabilities used in our work reflect more the likelihood of true events. We also applied SMOTE to the *k-nearest neigbour* algorithm inorder to address the dataset imbalance.

6.2 Future Work

Our results here were obtained from training models on a single dataset. As suggested by [10] the robustness of classification methods in credit scoring be examined better with more data and using more datasets can help in this regard. We mainly focused on popular machine learning learning methods and did not explore novel techniques which can also be incorporated in the foundational algorithms, for example our research did not cover much on convolutional neural networks and selective ensemble methods.

Evaluation of classification algorithms is another area that has received much attention recently and best practice suggests the use of different types of evaluation metrics. There are three broad types of evaluation metrics: threshold, ranking and probabilistic metrics. Most studies, including our own, have focused on threshold metrics (e.g. accuracy and F-measure), the ranking methods and metrics (e.g. ROC analysis and AUROC), and have left out other crucial measures of classifier performance.

Complex Machine Learning models present challenges in that it is hard to explain why a model produces some given outcomes. Research is gaining momentum therefore in ways to address this difficulty. Even though we did not explore some of the techniques which have been developed to give some insight into the internal logic of a model, further study can be done on techniques such as:

- Graphical tools to show the effect of an explanatory variable on the model. Here we employ Partial dependence plots to analyse the effect on the average prediction, while individual conditional expectations show the effect on a specific prediction.
- Feature importance measures to reveal the relevance of each explanatory variable in the overall model.
- Shapley values to quantify the impact of each explanatory variable on a specific prediction of the model.

The above are some of the many *white box* improvement techniques that are being proposed for credit scoring machine learning models.

Bibliography

[1] Ethem Alpaydin. *Introduction to Machine Learning (Adaptive Computation and Machine Learning)*. The MIT Press, 2004.

[2] Baesens Bart. *Developing Intelligent Systems for Credit Scoring Using Machine Learning Techniques*. PhD thesis, K.U.Leuven, Belgium, 2003.

[3] European Banking Authority, France. *EBA DISCUSSION PAPER ON MACHINE LEARNING FOR IRB MODELS*, November,11 2021 [Online]. URL: https://www.eba.europa.eu/sites/default/documents/files/document_library/Publications/Discussions/2022/Discussion%20on%20machine%20learning%20for%20IRB%20models/1023883/Discussion%20paper%20on%20machine%20learning%20for%20IRB%20models.pdf.

[4] Global Association of Risk Professionals, USA. *The Rise of Machine Learning in IRB Models: New EBA Outlook Could Open Door*, December,10 2021 [Online]. URL: https://www.garp.org/risk-intelligence/technology/the-rise-of-machine-learning-in-irb-models-new-eba-outlook-could-open-door.

[5] Terry Harris. Quantitative credit risk assessment using support vector machines: Broad versus narrow default definitions. *Expert Systems with Applications*, 40(11):4404–4413, 2013. URL: https://www.sciencedirect.com/science/article/pii/S0957417413000754, doi:https://doi.org/10.1016/j.eswa.2013.01.044.

[6] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer Series in Statistics. Springer New York Inc., New York, NY, USA, 2001.

[7] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2013. URL: https://faculty.marshall.usc.edu/gareth-james/ISL/.

[8] Mellisa L Knuston. *Credit Scoring Approaches guidelines*. World Bank Group, Washington, DC, USA, 2020 [Online]. URL: https://thedocs.worldbank.org/en/doc/935891585869698451-0130022020/CREDIT-SCORING-APPROACHES-GUIDELINES-FINAL-WEB.

[9] Sanjeev Kulkarni and Gilbert Harman. *An Elementary Introduction to Statistical Learning Theory*. Wiley Series in Probability and Statistics. John Wiley Sons, Inc., Hoboken, New Jersey, 2011.

[10] Stefan Lessmann, Bart Baesens, Hsin-Vonn Seow, and Lyn Thomas. Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, (doi:10.1016/j.ejor.2015.05.030), 05 2015. doi:10.1016/j.ejor.2015.05.030.

[11] R.L. Liu. Machine learning approaches to predict default of credit card clients. *Modern Economy*, 9:1828–1838, 2018. URL: https://doi.org/10.4236/me.2018.911115.

[12] Rashmi Malhotra and D. K. Malhotra. Evaluating consumer loans using neural networks. *Omega*, 31(2):83–96, 2003.

[13] Lyn Thomas, David Edelman, and Jonathan Crook. *Credit Scoring and its Applications*. 01 2002.

[14] Lien C. H. Yeh, I. C. The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2):2473–2480, 2009.

[15] Hu Y.P. Zhang, K.X. and Y.H. Wu. Classification and prediction on rural property mortgage data with three data mining methods. *Journal of Software Engineering and Applications*, 11:348–361, 2018. URL: https://doi.org/10.4236/jsea.2018.117022.

Appendix

Artificial Neural Network Code

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
get_ipython().run_line_magic('matplotlib', 'inline')


# In[2]:


df=pd.read_excel(r'credit_cards.xlsx',engine='openpyxl',header=1)


# In[3]:


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import keras
from keras.models import Sequential
from keras.layers import Dense,Activation
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import
    accuracy_score,precision_score,roc_auc_score,confusion_matrix,
                        recall_score,f1_score,brier_score_loss
from hmeasure import h_score
```

```
# In[4]:


X=df.drop('default payment next month',axis=1)
y=df['default payment next month']
scaler=StandardScaler()


# In[5]:


model=Sequential()
model.add(Dense(5, input_dim=24,activation='sigmoid',
kernel_initializer='he_uniform'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='sgd',metrics='AUC')


# In[6]:


def get_scores(X_train, X_test,y_train,y_test):
    clas=model.fit(scaled_X_train,y_train,class_weight={0:0.2212,1:0.7788},
                   epochs=300,verbose=0)
    y_pred=np.where(model.predict(scaled_X_test)>0.5,1,0)
    y_pred1=model.predict_proba(scaled_X_test)
    accuracy=accuracy_score(y_test,y_pred).round(2)
    auroc=roc_auc_score(y_test,y_pred)
    conf=confusion_matrix(y_test,y_pred)
    precision=precision_score(y_test,y_pred,pos_label=0)
    recall=recall_score(y_test,y_pred,pos_label=0)
    h=h_score(np.array(y_test),y_pred,pos_label=0)
    f1=f1_score(y_test,y_pred,pos_label=0)
    bs=brier_score_loss(y_test,y_pred1[:,0],pos_label=0)
    return accuracy,auroc,conf,precision,recall,f1,h,bs


# In[10]:


an_accuracy=[]
an_auroc=[]
an_conf=np.zeros([2,2])
an_precision=[]
an_recall=[]
an_f1=[]
an_h=[]
an_bs=[]

kfolds=10
```

```
skf=StratifiedKFold(n_splits=kfolds)
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    scaled_X_train=scaler.fit_transform(X_train)
    scaled_X_test=scaler.transform(X_test)



    #ANN Scores
    an_accuracy.append(get_scores(scaled_X_train,
        scaled_X_test,y_train,y_test)[0])
    an_auroc.append(get_scores(scaled_X_train,
        scaled_X_test,y_train,y_test)[1])
    an_conf+=get_scores(scaled_X_train, scaled_X_test,y_train,y_test)[2]
    an_precision.append(get_scores(scaled_X_train,
        scaled_X_test,y_train,y_test)[3])
    an_recall.append(get_scores(scaled_X_train,
        scaled_X_test,y_train,y_test)[4])
    an_f1.append(get_scores(scaled_X_train,
        scaled_X_test,y_train,y_test)[5])
    an_h.append(get_scores(scaled_X_train,
        scaled_X_test,y_train,y_test)[6])
    an_bs.append(get_scores(scaled_X_train,
        scaled_X_test,y_train,y_test)[7])


# Overall scores ANN
man_accuracy=(np.array(an_accuracy)).mean()
man_auroc=(np.array(an_auroc)).mean()
man_precision=(np.array(an_precision)).mean()
man_recall=(np.array(an_recall)).mean()
man_f1=(np.array(an_f1)).mean()
man_conf=(an_conf/kfolds).round()
man_h=(np.array(an_h)).mean()
man_bs=(np.array(an_bs)).mean()


# In[1]:


# Model Perfomance Metrics
print("%22s %14s %14s %14s %14s %14s %14s %14s" % ('Model', 'Accuracy',
    'AUROC','Precision','Recall','F1','H-measure','Brier Score'))
print(63 * "-+")
print("%22s %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f" % ('ANN',
    man_accuracy, man_auroc,
    man_precision,man_recall,man_f1,man_h,man_bs))
```

```
# In[11]:


# Confusion Matrix
man_conf


# In[12]:


#Brier Skill Score

bss=1-(man_bs/1).round(3)
bss


# In[11]:


y_pred


# In[ ]:
```

---

kNN Code

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
get_ipython().run_line_magic('matplotlib', 'inline')


# In[2]:


df=pd.read_excel(r'credit_cards.xlsx',engine='openpyxl',header=1)
```

```
# In[6]:


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import
    accuracy_score,precision_score,roc_auc_score,confusion_matrix,
                        recall_score,f1_score,brier_score_loss
from sklearn.calibration import CalibratedClassifierCV
from hmeasure import h_score


# In[4]:


X=df.drop('default payment next month',axis=1)
y=df['default payment next month']
scaler=StandardScaler()


# In[7]:


kn=KNeighborsClassifier(n_neighbors=10,metric='euclidean')
# Calibrate the model
c_kn = CalibratedClassifierCV(kn, method='isotonic', cv=3)
sm=SMOTE(random_state=101)


# In[8]:


def get_scores(classifier,X_train, X_test,y_train,y_test):
    classifier.fit(X_resamp,y_resamp)
    y_pred=classifier.predict(scaled_X_test)
    y_pred1=classifier.predict_proba(scaled_X_test)
    accuracy=accuracy_score(y_test,y_pred).round(2)
    auroc=roc_auc_score(y_test,y_pred)
    conf=confusion_matrix(y_test,y_pred)
    precision=precision_score(y_test,y_pred,pos_label=0)
    recall=recall_score(y_test,y_pred,pos_label=0)
    h=h_score(np.array(y_test),y_pred,pos_label=0)
    f1=f1_score(y_test,y_pred,pos_label=0)
    bs=brier_score_loss(y_test,y_pred1[:,0],pos_label=0)
    return accuracy,auroc,conf,precision,recall,f1,h,bs
```

```
# In[9]:


# kNN sccore lists
kn_accuracy=[]
kn_auroc=[]
kn_conf=np.zeros([2,2])
kn_precision=[]
kn_recall=[]
kn_f1=[]
kn_h=[]
kn_bs=[]

kfolds=10

skf=StratifiedKFold(n_splits=kfolds)
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    scaled_X_train=scaler.fit_transform(X_train)
    scaled_X_test=scaler.transform(X_test)

    X_resamp,y_resamp=sm.fit_resample(scaled_X_train,y_train)


    # update kNN Scores
    kn_accuracy.append(get_scores(c_kn,X_resamp,
        scaled_X_test,y_resamp,y_test)[0])
    kn_auroc.append(get_scores(c_kn,X_resamp,
        scaled_X_test,y_resamp,y_test)[1])
    kn_conf+=get_scores(c_kn,X_resamp, scaled_X_test,y_resamp,y_test)[2]
    kn_precision.append(get_scores(c_kn,X_resamp,
        scaled_X_test,y_resamp,y_test)[3])
    kn_recall.append(get_scores(c_kn,X_resamp,
        scaled_X_test,y_resamp,y_test)[4])
    kn_f1.append(get_scores(c_kn,X_resamp,
        scaled_X_test,y_resamp,y_test)[5])
    kn_h.append(get_scores(c_kn,X_resamp,
        scaled_X_test,y_resamp,y_test)[6])
    kn_bs.append(get_scores(c_kn,X_resamp,
        scaled_X_test,y_resamp,y_test)[7])


# Overall scores kNN
mkn_accuracy=(np.array(kn_accuracy)).mean()
mkn_auroc=(np.array(kn_auroc)).mean()
mkn_precision=(np.array(kn_precision)).mean()
mkn_recall=(np.array(kn_recall)).mean()
mkn_f1=(np.array(kn_f1)).mean()
```

```
mkn_conf=(kn_conf/kfolds).round()
mkn_h=(np.array(kn_h)).mean()
mkn_bs=(np.array(kn_bs)).mean()


# In[10]:


# Model Perfomance Metrics
print("%22s %14s %14s %14s %14s %14s %14s %14s" % ('Model', 'Accuracy',
    'AUROC','Precision','Recall','F1','H-measure','Brier Score'))
print(63 * "-+")
print("%22s %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f" % ('KNN',
    mkn_accuracy, mkn_auroc,
    mkn_precision,mkn_recall,mkn_f1,mkn_h,mkn_bs))


# In[11]:


mkn_conf


# In[19]:


#Brier Skill Score

bss=(1-(mkn_bs/1)).round(3)
bss


# In[ ]:
```

SVM, Logistic Regression, Decision Tree and Random Forest Code

---

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns
get_ipython().run_line_magic('matplotlib', 'inline')


# In[2]:


df=pd.read_excel(r'credit_cards.xlsx',engine='openpyxl',header=1)


# In[3]:


X=df.drop('default payment next month',axis=1)
y=df['default payment next month']


# In[4]:


from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import
    accuracy_score,precision_score,roc_auc_score,confusion_matrix,
                        recall_score,f1_score,brier_score_loss
from sklearn.preprocessing import StandardScaler
from sklearn.calibration import CalibratedClassifierCV
from hmeasure import h_score


# In[5]:


scaler=StandardScaler()

# models with optimized hyperparameters

dt=DecisionTreeClassifier(criterion='entropy', max_depth=6,
    min_samples_leaf=7,
                            min_samples_split=12,splitter='best',\
                        class_weight={0:0.2212,1:0.7788})
lr=LogisticRegression(solver='newton-cg',class_weight={0:0.2212,1:0.7788})
rf=RandomForestClassifier(class_weight='balanced',n_estimators=250,
                    max_features=3,criterion='entropy',
                        bootstrap=True,oob_score=True)
svm=SVC(C=3.0,kernel='rbf',random_state=0,class_weight={0: 0.2212, 1:
    0.7788})
```

```
# In[6]:


# Calibrate the models
c_dt = CalibratedClassifierCV(dt, method='isotonic', cv=3)
c_lr = CalibratedClassifierCV(lr, method='isotonic', cv=3)
c_rf = CalibratedClassifierCV(rf, method='isotonic', cv=3)
c_svm = CalibratedClassifierCV(svm, method='isotonic', cv=3)


# In[7]:


def get_scores(classifier,X_train, X_test,y_train,y_test):
    classifier.fit(scaled_X_train,y_train)
    y_pred=classifier.predict(scaled_X_test)
    y_pred1=classifier.predict_proba(scaled_X_test)
    accuracy=accuracy_score(y_test,y_pred).round(2)
    auroc=roc_auc_score(y_test,y_pred)
    conf=confusion_matrix(y_test,y_pred)
    precision=precision_score(y_test,y_pred,pos_label=0)
    recall=recall_score(y_test,y_pred,pos_label=0)
    h=h_score(np.array(y_test),y_pred,pos_label=0)
    f1=f1_score(y_test,y_pred,pos_label=0)
    bs=brier_score_loss(y_test,y_pred1[:,0],pos_label=0)
    return accuracy,auroc,conf,precision,recall,f1,h,bs


# In[8]:


# Decision tree lists
dt_accuracy=[]
dt_auroc=[]
dt_conf=np.zeros([2,2])
dt_precision=[]
dt_recall=[]
dt_f1=[]
dt_h=[]
dt_bs=[]

#logistic regression lists
lr_accuracy=[]
lr_auroc=[]
lr_conf=np.zeros([2,2])
lr_precision=[]
lr_recall=[]
lr_f1=[]
```

```
lr_h=[]
lr_bs=[]

# rf lists
rf_accuracy=[]
rf_auroc=[]
rf_conf=np.zeros([2,2])
rf_precision=[]
rf_recall=[]
rf_f1=[]
rf_h=[]
rf_bs=[]

#Svm lists
svm_accuracy=[]
svm_auroc=[]
svm_conf=np.zeros([2,2])
svm_precision=[]
svm_recall=[]
svm_f1=[]
svm_h=[]
svm_bs=[]



kfolds=10

skf=StratifiedKFold(n_splits=kfolds)
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    scaled_X_train=scaler.fit_transform(X_train)
    scaled_X_test=scaler.transform(X_test)

    #Decision Tree Scores
    dt_accuracy.append(get_scores(c_dt,scaled_X_train,
        scaled_X_test,y_train,y_test)[0])
    dt_auroc.append(get_scores(c_dt,scaled_X_train,
        scaled_X_test,y_train,y_test)[1])
    dt_conf+=get_scores(c_dt,scaled_X_train,
        scaled_X_test,y_train,y_test)[2]
    dt_precision.append(get_scores(c_dt,scaled_X_train,
        scaled_X_test,y_train,y_test)[3])
    dt_recall.append(get_scores(c_dt,scaled_X_train,
        scaled_X_test,y_train,y_test)[4])
    dt_f1.append(get_scores(c_dt,scaled_X_train,
        scaled_X_test,y_train,y_test)[5])
    dt_h.append(get_scores(c_dt,scaled_X_train,
        scaled_X_test,y_train,y_test)[6])
```

```
dt_bs.append(get_scores(c_dt,scaled_X_train,
    scaled_X_test,y_train,y_test)[7])


# logistic regression scores
lr_accuracy.append(get_scores(c_lr,scaled_X_train,
    scaled_X_test,y_train,y_test)[0])
lr_auroc.append(get_scores(c_lr,scaled_X_train,
    scaled_X_test,y_train,y_test)[1])
lr_conf+=get_scores(c_lr,scaled_X_train,
    scaled_X_test,y_train,y_test)[2]
lr_precision.append(get_scores(c_lr,scaled_X_train,
    scaled_X_test,y_train,y_test)[3])
lr_recall.append(get_scores(c_lr,scaled_X_train,
    scaled_X_test,y_train,y_test)[4])
lr_f1.append(get_scores(c_lr,scaled_X_train,
    scaled_X_test,y_train,y_test)[5])
lr_h.append(get_scores(c_lr,scaled_X_train,
    scaled_X_test,y_train,y_test)[6])
lr_bs.append(get_scores(c_lr,scaled_X_train,
    scaled_X_test,y_train,y_test)[7])

# Random Forest scores
rf_accuracy.append(get_scores(c_rf,scaled_X_train,
    scaled_X_test,y_train,y_test)[0])
rf_auroc.append(get_scores(c_rf,scaled_X_train,
    scaled_X_test,y_train,y_test)[1])
rf_conf+=get_scores(c_rf,scaled_X_train,
    scaled_X_test,y_train,y_test)[2]
rf_precision.append(get_scores(c_rf,scaled_X_train,
    scaled_X_test,y_train,y_test)[3])
rf_recall.append(get_scores(c_rf,scaled_X_train,
    scaled_X_test,y_train,y_test)[4])
rf_f1.append(get_scores(c_rf,scaled_X_train,
    scaled_X_test,y_train,y_test)[5])
rf_h.append(get_scores(c_rf,scaled_X_train,
    scaled_X_test,y_train,y_test)[6])
rf_bs.append(get_scores(c_rf,scaled_X_train,
    scaled_X_test,y_train,y_test)[7])

# Support Vector Machine scores
svm_accuracy.append(get_scores(c_svm,scaled_X_train,
    scaled_X_test,y_train,y_test)[0])
svm_auroc.append(get_scores(c_svm,scaled_X_train,
    scaled_X_test,y_train,y_test)[1])
svm_conf+=get_scores(c_svm,scaled_X_train,
    scaled_X_test,y_train,y_test)[2]
svm_precision.append(get_scores(c_svm,scaled_X_train,
    scaled_X_test,y_train,y_test)[3])
```

```
    svm_recall.append(get_scores(c_svm,scaled_X_train,
        scaled_X_test,y_train,y_test)[4])
    svm_f1.append(get_scores(c_svm,scaled_X_train,
        scaled_X_test,y_train,y_test)[5])
    svm_h.append(get_scores(c_svm,scaled_X_train,
        scaled_X_test,y_train,y_test)[6])
    svm_bs.append(get_scores(c_svm,scaled_X_train,
        scaled_X_test,y_train,y_test)[7])


# Overall scores Decision Tree
mdt_accuracy=(np.array(dt_accuracy)).mean()
mdt_auroc=(np.array(dt_auroc)).mean()
mdt_precision=(np.array(dt_precision)).mean()
mdt_recall=(np.array(dt_recall)).mean()
mdt_f1=(np.array(dt_f1)).mean()
mdt_conf=(dt_conf/kfolds).round()
mdt_h=(np.array(dt_h)).mean()
mdt_bs=(np.array(dt_bs)).mean()

# overall scores logistic regression
mlr_accuracy=(np.array(lr_accuracy)).mean()
mlr_auroc=(np.array(lr_auroc)).mean()
mlr_precision=(np.array(lr_precision)).mean()
mlr_recall=(np.array(lr_recall)).mean()
mlr_f1=(np.array(lr_f1)).mean()
mlr_conf=(lr_conf/kfolds).round()
mlr_h=(np.array(lr_h)).mean()
mlr_bs=(np.array(lr_bs)).mean()

# Overal scores Random Forest
mrf_accuracy=(np.array(rf_accuracy)).mean()
mrf_auroc=(np.array(rf_auroc)).mean()
mrf_precision=(np.array(rf_precision)).mean()
mrf_recall=(np.array(rf_recall)).mean()
mrf_f1=(np.array(rf_f1)).mean()
mrf_conf=(rf_conf/kfolds).round()
mrf_h=(np.array(rf_h)).mean()
mrf_bs=(np.array(rf_bs)).mean()

# Overal scores Support Vector Machine
msvm_accuracy=(np.array(svm_accuracy)).mean()
msvm_auroc=(np.array(svm_auroc)).mean()
msvm_precision=(np.array(svm_precision)).mean()
msvm_recall=(np.array(svm_recall)).mean()
msvm_f1=(np.array(svm_f1)).mean()
msvm_conf=(svm_conf/kfolds).round()
msvm_h=(np.array(svm_h)).mean()
msvm_bs=(np.array(svm_bs)).mean()
```

```
# In[9]:


# Model Perfomance Metrics
print("%22s %14s %14s %14s %14s %14s %14s %14s" % ('Model', 'Accuracy',
    'AUROC','Precision','Recall','F1','H-measure','Brier Score'))
print(63 * "-+")
print("%22s %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f" %
    ('Decision Tree', mdt_accuracy, mdt_auroc,

    mdt_precision,mdt_recall,mdt_f1,mdt_h,mdt_bs))
print("%22s %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f" %
    ('Logistic Regression', mlr_accuracy, mlr_auroc,

    mlr_precision,mlr_recall,mlr_f1,mlr_h,mlr_bs))
print("%22s %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f" %
    ('Support Vector Machine', msvm_accuracy, msvm_auroc,

    msvm_precision,msvm_recall,msvm_f1,msvm_h,msvm_bs))
print("%22s %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f %14.3f" % ('Random
    Forest', mrf_accuracy, mrf_auroc,
    mrf_precision,mrf_recall,mrf_f1,mrf_h,mrf_bs))
print(63 * "-+")


# In[10]:


mdt_conf


# In[11]:


mlr_conf


# In[12]:


msvm_conf


# In[13]:


mrf_conf
```

```
# In[14]:


#Brier Skill Score Decision Tree

bss=(1-(mdt_bs/1)).round(3)
bss


# In[15]:


#Brier Skill Score Logistic Regression

bssl=(1-(mlr_bs/1)).round(3)
bssl


# In[16]:


#Brier Skill Score Support Vector Machine

bsss=(1-(msvm_bs/1)).round(3)
bsss


# In[17]:


#Brier Skill Score Random Forest

bssr=(1-(mrf_bs/1)).round(3)
bssr


# In[ ]:
```